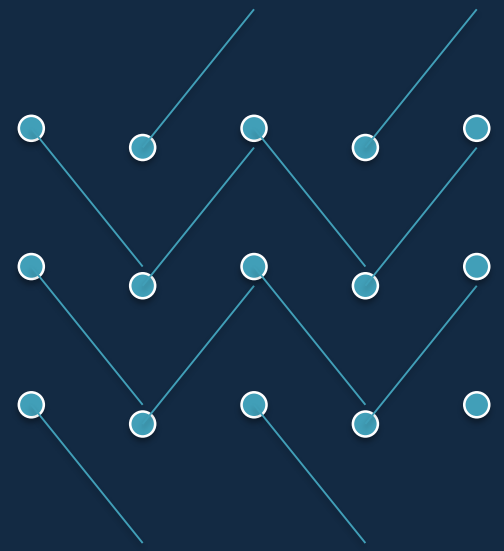
NEURAL NETWORK COMPONENTS



Structure, functions, and practical experimentation
with TensorFlow Neural Network Playground

Portfolio Artifact | Workshop 2.4

Prepared by Soujanya Pasumarthi

From neural-network theory to visible model behavior

This artifact defines the required components and connects them to observations from hands-on model training.

01

DEFINE

Explain layers, neurons, weights, activation functions, loss functions, and optimization algorithms.

02

ILLUSTRATE

Trace information from input features through hidden layers to an output prediction.

03

EXPERIMENT

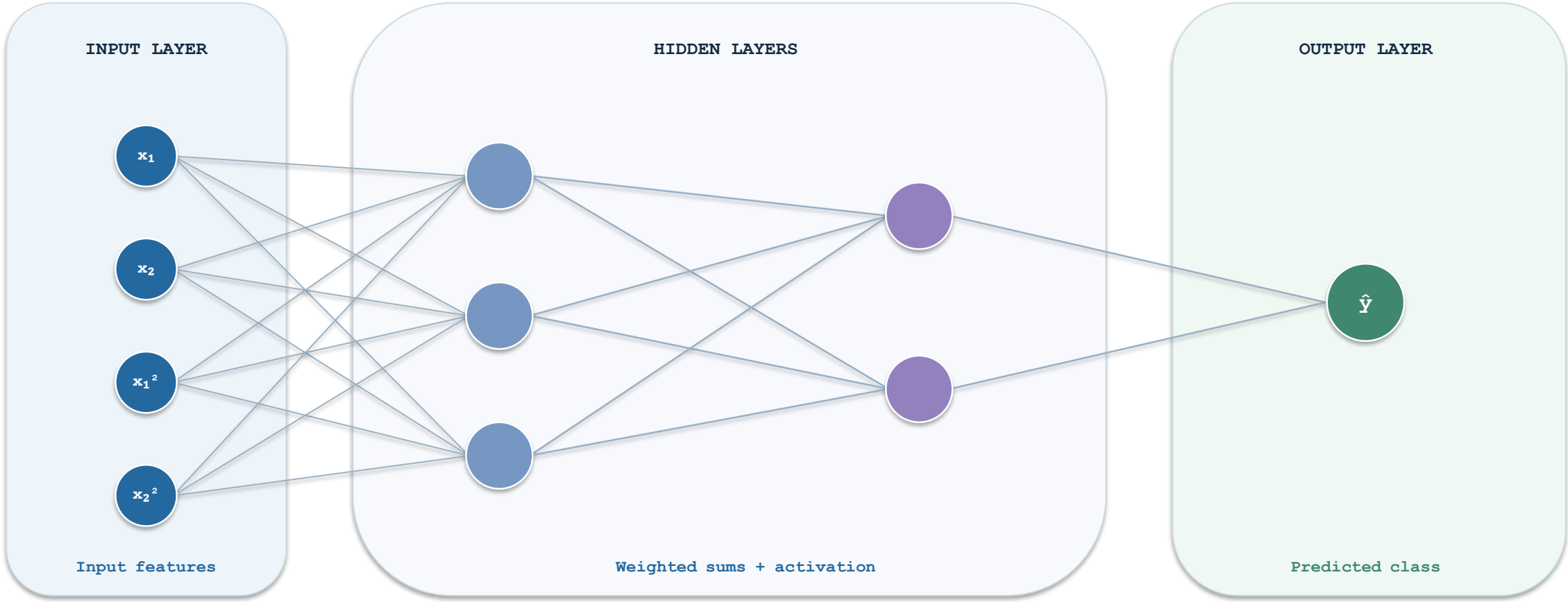
Change features, noise, activation, and architecture to observe model behavior.

EXPERIMENTAL LENS

TensorFlow Playground was used to train classification networks on the Circle and XOR datasets. The captured experiments demonstrate how changes in network structure, activation function, learning rate, and noise affect the model's learned decision boundary and its reported training and test loss.

How information moves through a neural network

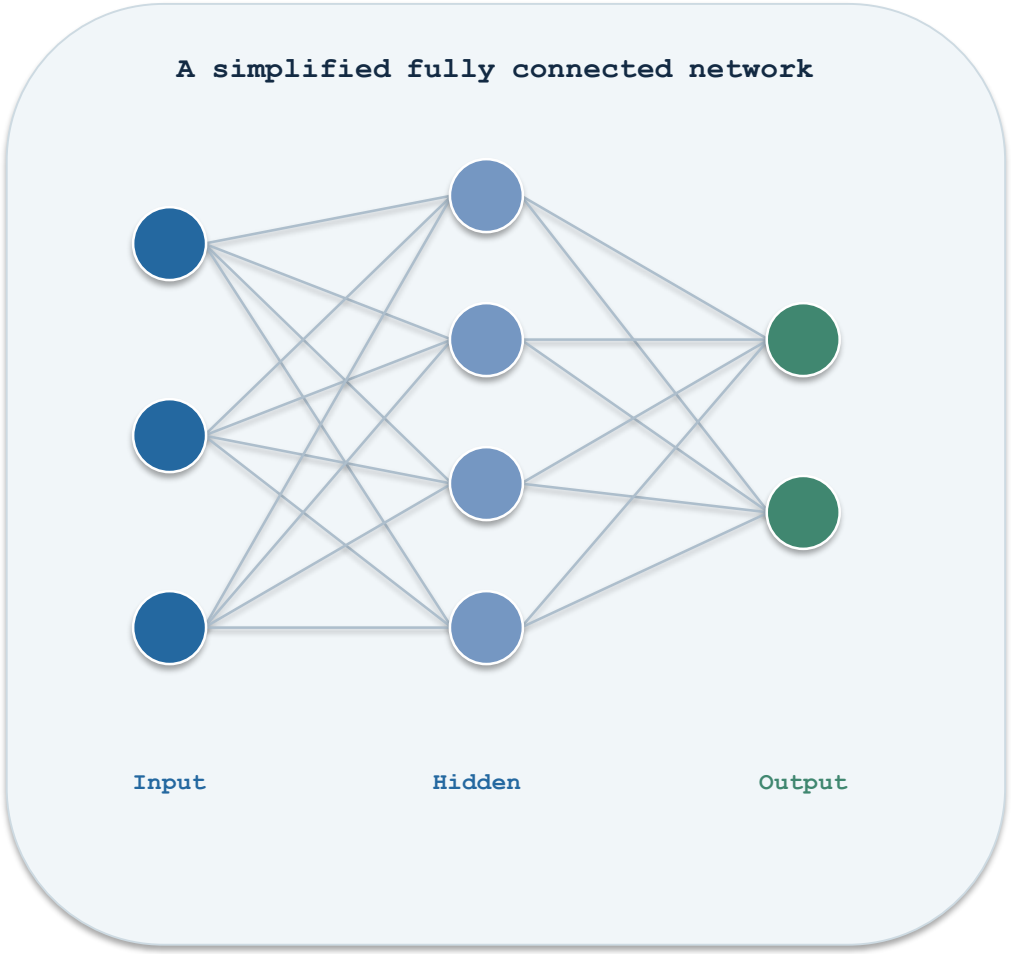
Forward propagation transforms input features into an output through weighted connections and activation functions.



The number of hidden layers and neurons is a design choice. Connections carry learned weights; the output represents the model's prediction.

Layers, neurons, and weights

These elements define the network’s structure and how signals are combined.



- **Layers**

  Groups of neurons arranged by role: input, one or more hidden layers, and output.

- **Neurons**

  Computational units that receive values, combine them, apply an activation, and pass a result forward.

- **Weights**

  Learnable numerical parameters on connections. Training adjusts weights to reduce prediction error.

More connections increase capacity, but do not guarantee better generalization.

Activation functions introduce nonlinearity

Without nonlinear activations, stacking linear layers remains equivalent to a linear transformation.

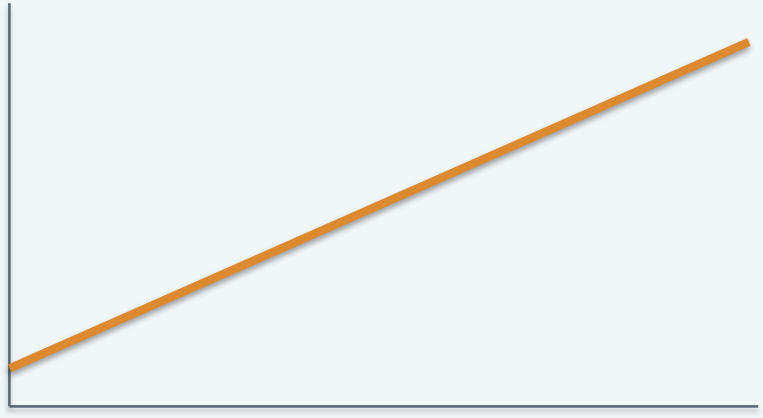
ReLU



f(x) = max(0, x)
Negative values become zero; positive values pass through.

NONLINEAR

Linear activation



f(x) = x
The input passes through unchanged; no nonlinearity is added.

LINEAR

Activation choice affects the boundary the model can learn.

Loss functions and optimization algorithms

A model learns by measuring prediction error and updating its parameters to reduce that error.

PREDICTION

$\hat{y} = \text{model}(x)$

Network produces an output.

LOSS

$L(y, \hat{y})$

Compare prediction with target.

GRADIENT

$\nabla L(w)$

Estimate how weights affect loss.

UPDATE

$w \leftarrow w - \alpha \nabla L$

Adjust weights using a learning rate.

LOSS FUNCTION

Quantifies the mismatch between predicted and actual values. The training objective is to minimize this quantity.

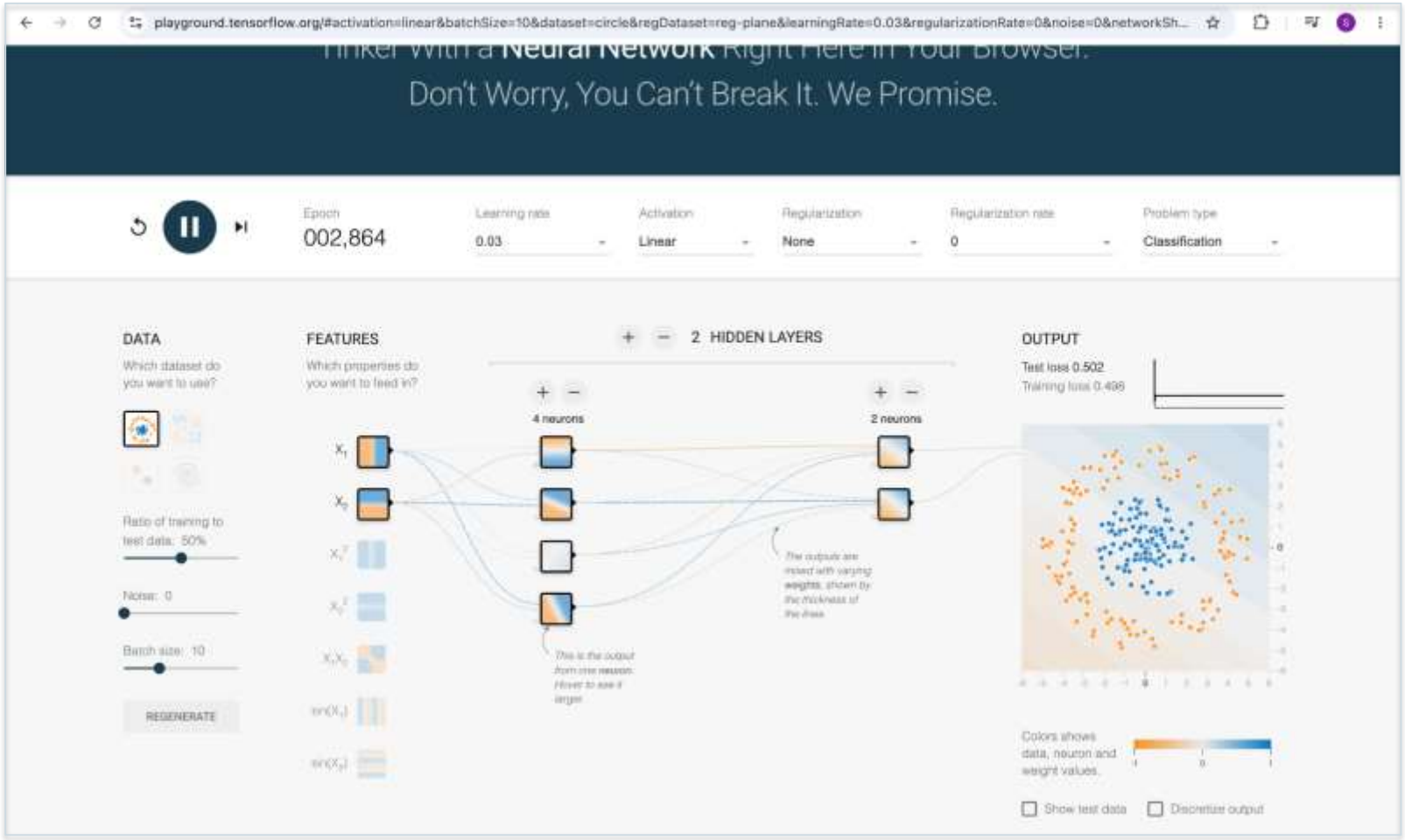
OPTIMIZATION ALGORITHM

Uses gradients to update weights. Gradient descent and adaptive methods such as Adam are common examples.

Training loss is measured on training data; test loss evaluates performance on held-out data.

TensorFlow Playground: experimental environment

The interface exposes data, features, hidden-layer design, activation, regularization, and training behavior.



Screenshot shows the initial Playground configuration; loss values are specific to this run and epoch.

What was controlled?

- **Dataset**

  Circle classification.

- **Features**

  x_1, x_2 and optional engineered terms.

- **Network**

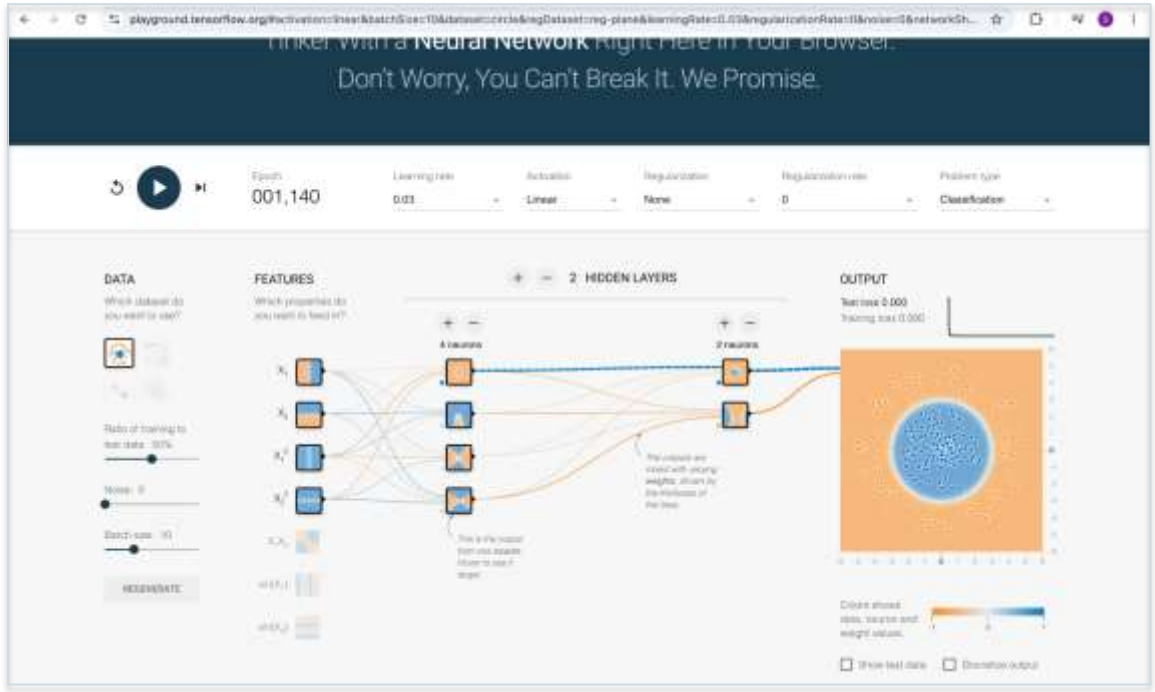
  Hidden layers and neuron counts.

- **Training**

  Learning rate, activation, noise, regularization.

Input representation and network structure matter

The supplied runs show that engineered features and layer configuration affect how the network represents the circle pattern.



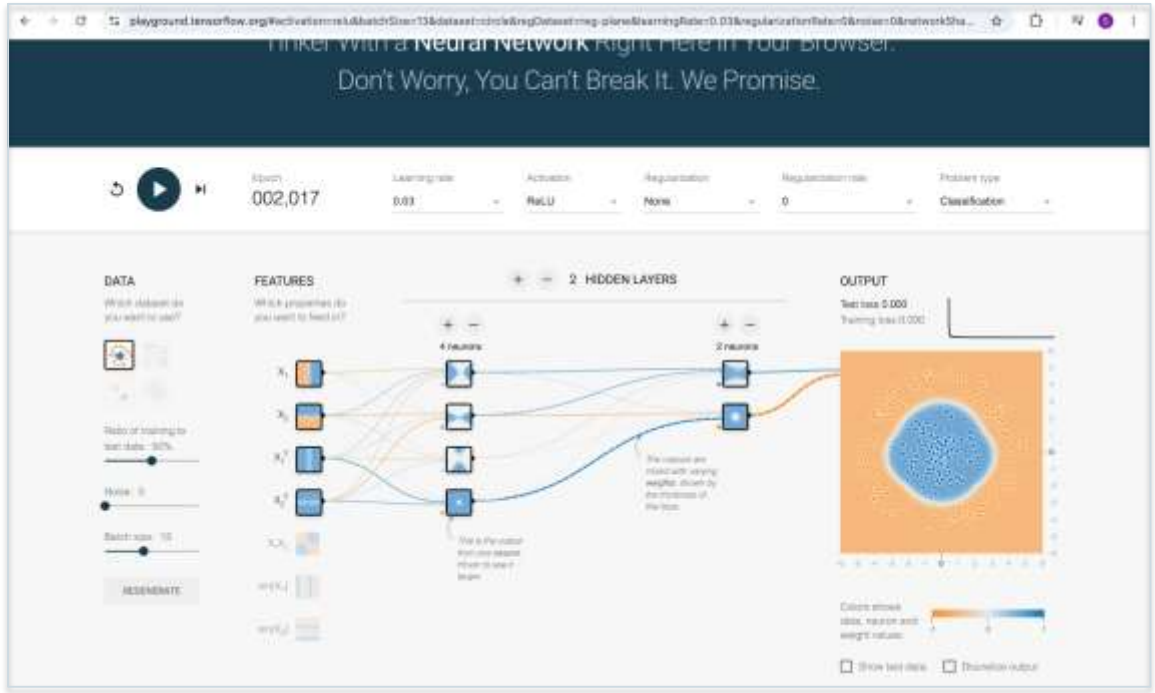


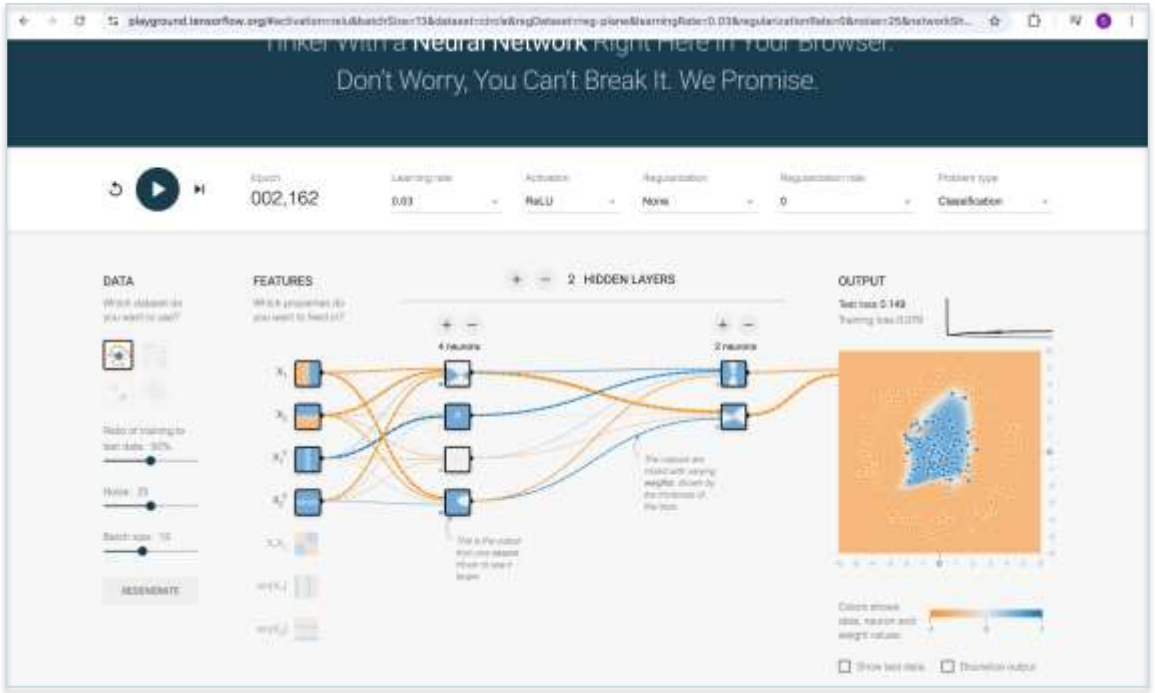
Squared inputs (x_1^2, x_2^2) add radial information that can help represent a circular boundary.

Changing layer and neuron counts changes capacity; this screenshot is an early training state.

Noise makes classification more difficult

Compare the two ReLU runs with learning rate 0.03 and the same two-hidden-layer layout; the noise setting differs.





NOISE = 0 **Test loss 0.000 | Training loss 0.000**

NOISE = 25 **Test loss 0.149 | Training loss 0.079**

Observation: with noise added, the learned boundary is less clean and displayed losses are higher.

ReLU and Linear produce different learned boundaries

These screenshots use the circle dataset with noise 25, learning rate 0.03, and the same displayed network layout.





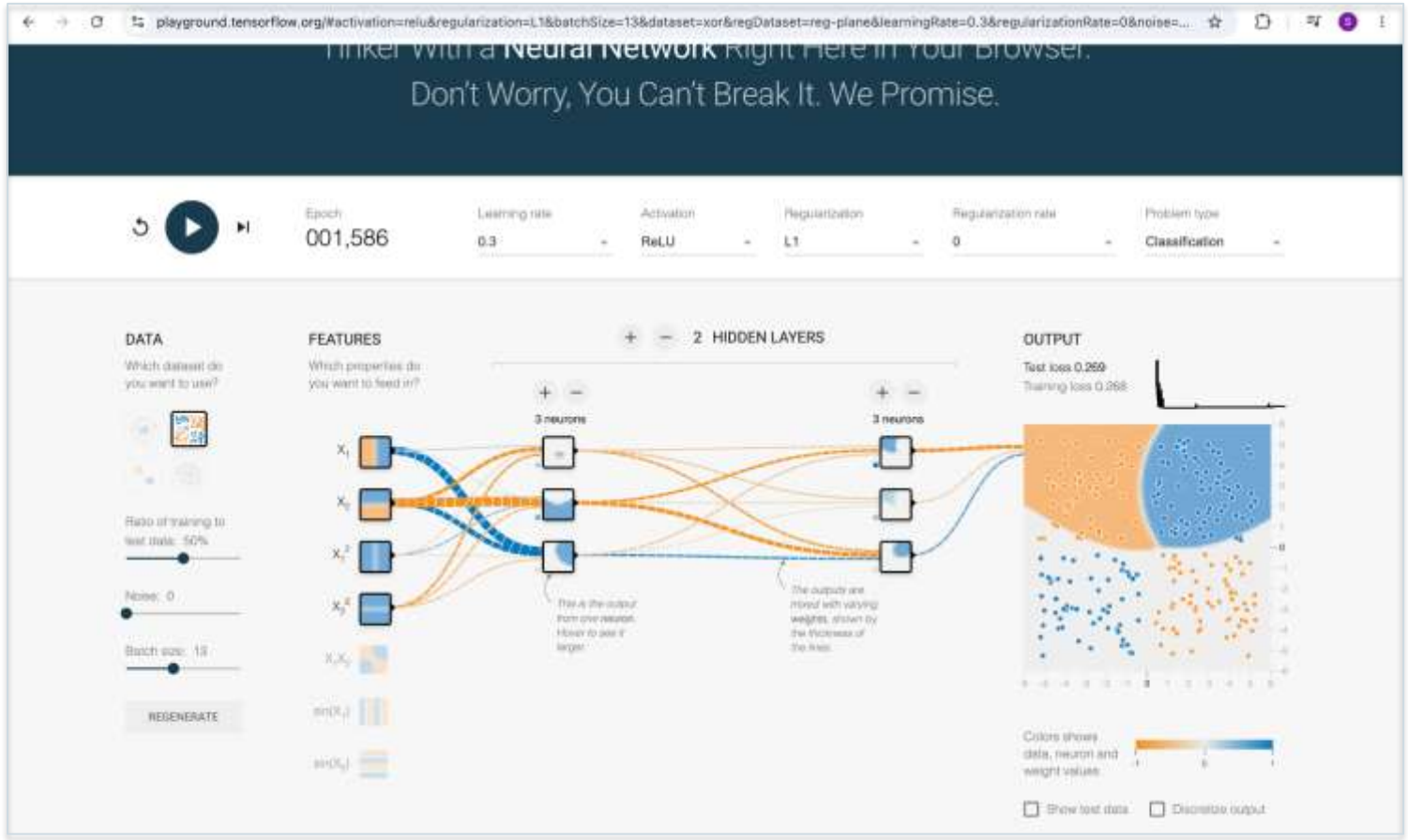
ReLU **Test 0.149 | Train 0.079**

LINEAR **Test 0.109 | Train 0.110**

A single run does not prove Linear is generally better; repeated trials under controlled conditions would be needed.

A separate example: XOR classification

This screenshot uses the XOR dataset, not the circle dataset; it illustrates a different classification pattern.



XOR run

Test loss 0.269
Training loss 0.268

The XOR pattern is not separable by a single straight decision boundary. Hidden units can combine inputs to form a nonlinear classification region.

Interpretation is limited to the settings and epoch shown.

CONCLUSION

Key insights from the neural-network experiments

Visualizing the network made the relationship between model structure, training choices, and classification behavior easier to interpret.

STRUCTURE	Layers organize computation; neurons transform inputs; weights determine connection strength.
LEARNING	Activation functions shape what patterns can be represented; loss measures error; optimization updates weights.
DATA	Feature representation and noise influence how clearly classes can be separated.
EVALUATION	Training and test loss provide complementary views; compare controlled runs and avoid conclusions from one trial.